Laporan Tugas Mandiri GEE

Nama :Hizba Shafwatuddin
NIM :241524046
Kelas :5B-D4

Tugas Mandiri

1. Sensitivitas clustering

```
/**************************************************************
 EKSPERIMEN K-MEANS LANDSAT 9
 K = 5, 10, 20
 5 pengulangan setiap K
 Kandidat lahan terbangun = cluster 2 dan 3
**************************************************************/

// 1. Wilayah kajian
var geometry2 = ee.Geometry.Polygon(
  [[[111.56510661311408, -7.125131151145682],
    [111.56510661311408, -7.178102498857165],
    [111.62192652888557, -7.178102498857165],
    [111.62192652888557, -7.125131151145682]]],
  null, false
);

Map.centerObject(geometry2, 12);
Map.addLayer(geometry2, {color: 'yellow'}, 'Wilayah kajian');

// 2. Ambil citra Landsat 9
var dataset = ee.ImageCollection('LANDSAT/LC09/C02/T1_L2')
  .filterBounds(geometry2)
  .filterMetadata('CLOUD_COVER_LAND', 'less_than', 50)
  .sort('CLOUD_COVER_LAND');

print('Jumlah citra tersedia:', dataset.size());
print('Citra pertama:', dataset.first());

// 3. Pilih citra
var img = ee.Image(dataset.first());

// Band reflektansi
var bands = [
  'SR_B1', 'SR_B2', 'SR_B3', 'SR_B4',
  'SR_B5', 'SR_B6', 'SR_B7'
];

var img_pilih = img.select(bands).clip(geometry2);
```

```
Map.addLayer(img_pilih, {
  bands: ['SR_B4', 'SR_B3', 'SR_B2'],
  min: 7061,
  max: 14103,
  gamma: 1
}, 'Landsat 9 RGB');

// 4. Buat sampel pelatihan tetap
var data_training = img_pilih.sample({
  region: geometry2,
  scale: 30,
  numPixels: 500,
  seed: 42,
  geometries: false
});

print('Sampel pelatihan:', data_training.size());

// 5. Parameter eksperimen
var jumlahCluster = [5, 10, 20];
var jumlahUlangan = 5;

// 6. Pelatihan berulang dan perhitungan luas
var hasilEksperimen = [];

jumlahCluster.forEach(function(k) {

  for (var r = 0; r < jumlahUlangan; r++) {

    // Seed berbeda untuk inisialisasi K-means.
    // Parameter lain dipertahankan tetap.
    var seed = 100 + r;

    var clusterer = ee.Clusterer.wekaKMeans(
      k,       // jumlah cluster
      0,       // metode inisialisasi
      false,   // canopies
      100,     // maxCandidates
      10000,   // periodicPruning
      2,       // minDensity
      -1.5,    // t1
      -1.0,    // t2
      'Euclidean',
      500,     // maxIterations
      false,   // preserveOrder
      false,   // fast
```

```
    seed     // seed inisialisasi
  ).train(data_training);

  // Terapkan model pada citra
  var clustered = img_pilih.cluster(clusterer);

  // Kandidat kelas terbangun: cluster 2 atau 3
  var kandidat = clustered.eq(2)
    .or(clustered.eq(3))
    .selfMask();

  // Luas setiap piksel dalam meter persegi
  var areaImage = ee.Image.pixelArea()
    .rename('area')
    .updateMask(kandidat);

  // Jumlahkan luas kandidat dalam wilayah kajian
  var areaDict = areaImage.reduceRegion({
    reducer: ee.Reducer.sum(),
    geometry: geometry2,
    scale: 30,
    maxPixels: 1e9,
    tileScale: 4
  });

  var areaM2 = ee.Number(
    ee.Algorithms.If(
      areaDict.get('area'),
      areaDict.get('area'),
      0
    )
  );

  // Simpan hasil setiap pengulangan
  hasilEksperimen.push(ee.Feature(null, {
    'K': k,
    'ulangan': r + 1,
    'seed': seed,
    'luas_m2': areaM2,
    'luas_ha': areaM2.divide(10000),
    'luas_km2': areaM2.divide(1e6)
  }));

  // Tampilkan hasil pengulangan pertama tiap K
  if (r === 0) {
    Map.addLayer(
```

```
        kandidat,
        {palette: ['ff7070']},
        'K=' + k + ', seed=' + seed
      );
    }
  }
});

// 7. Tampilkan tabel hasil
var tabel = ee.FeatureCollection(hasilEksperimen)
  .sort('K');

print('Tabel luas kandidat lahan terbangun:', tabel);

// 8. Ringkasan statistik luas untuk setiap K
var ringkasan = ee.FeatureCollection(
  jumlahCluster.map(function(k) {

    var subset = tabel.filter(ee.Filter.eq('K', k));

    var statistik = subset.reduceColumns({
      reducer: ee.Reducer.mean()
        .combine({
          reducer2: ee.Reducer.stdDev(),
          sharedInputs: true
        }),
      selectors: ['luas_ha']
    });

    return ee.Feature(null, {
      'K': k,
      'rata_luas_ha': statistik.get('mean'),
      'sd_luas_ha': statistik.get('stdDev'),
      'jumlah_ulangan': subset.size()
    });
  })
);

print('Ringkasan luas per jumlah cluster:', ringkasan);
```



Kode ini digunakan untuk melakukan eksperimen klasifikasi citra satelit Landsat 9 menggunakan algoritma K-Means pada wilayah kajian yang telah ditentukan. Pertama, kode mengambil citra Landsat 9 dengan tutupan awan terendah dari kumpulan citra yang memenuhi kriteria, kemudian memilih tujuh band reflektansi untuk dianalisis. Selanjutnya, kode mengambil 500 sampel piksel sebagai data pelatihan yang digunakan secara konsisten dalam seluruh eksperimen.

Eksperimen dilakukan dengan tiga variasi jumlah cluster, yaitu \\(K=5\\), \\(K=10\\), dan \\(K=20\\), dengan masing-masing nilai K diulang sebanyak lima kali menggunakan seed yang berbeda. Tujuannya adalah mengamati variasi hasil pengelompokan piksel berdasarkan karakteristik spektral citra. Setelah proses pengelompokan, cluster nomor 2 dan 3 dipilih sebagai kandidat lahan terbangun, kemudian luas gabungan kedua cluster tersebut dihitung dalam satuan meter persegi, hektare, dan kilometer persegi. Namun, pemilihan kedua cluster tersebut masih merupakan asumsi yang perlu divalidasi.

Terakhir, kode menyajikan hasil setiap pengulangan dalam bentuk tabel serta menghitung rata-rata dan standar deviasi luas kandidat lahan terbangun untuk setiap nilai K. Hasil ini dapat digunakan untuk membandingkan pengaruh jumlah cluster terhadap estimasi luas serta mengevaluasi konsistensi hasil antarulangan. Perlu diperhatikan bahwa kestabilan luas yang dihasilkan tidak secara langsung menunjukkan akurasi klasifikasi, karena identifikasi cluster sebagai lahan terbangun tetap memerlukan pemeriksaan dan validasi lebih lanjut.

2. Kalibrasi ambangan

```
/**********************************************************
 KALIBRASI EKSPERIMENTAL NDBI
 5 kandidat terbangun + 5 kandidat bukan terbangun
 Pembanding ambang NDBI awal = -0.1
```

```
**********************************************************/

// 1. ROI ASLI
var roi = ee.Geometry({
  geodesic: false,
  type: 'Polygon',
  coordinates: [[
    [106.53598136542733, -6.6221756519036195],
    [107.23910636542733, -6.6221756519036195],
    [107.23910636542733, -5.928728445061094],
    [106.53598136542733, -5.928728445061094],
    [106.53598136542733, -6.6221756519036195]
  ]]
});

// 2. LANDSAT 8 COLLECTION 2 LEVEL 2
var l8 = ee.ImageCollection('LANDSAT/LC08/C02/T1_L2');

var filtered = l8
  .filterBounds(roi)
  .filterDate('2022-01-01', '2023-01-01');

print('Jumlah citra sebelum masking:', filtered.size());

// 3. MASKING AWAN DAN SKALA REFLEKTANSI
function cloudMask(image) {
  var qa = image.select('QA_PIXEL');

  var dilated = 1 << 1;
  var cirrus = 1 << 2;
  var cloud = 1 << 3;
  var shadow = 1 << 4;

  var mask = qa.bitwiseAnd(dilated).eq(0)
    .and(qa.bitwiseAnd(cirrus).eq(0))
    .and(qa.bitwiseAnd(cloud).eq(0))
    .and(qa.bitwiseAnd(shadow).eq(0));

  var scaled = image.select(
    ['SR_B2', 'SR_B3', 'SR_B4',
     'SR_B5', 'SR_B6', 'SR_B7'],
    ['B2', 'B3', 'B4', 'B5', 'B6', 'B7']
  ).multiply(0.0000275).add(-0.2);

  return scaled.updateMask(mask)
    .copyProperties(image, image.propertyNames());
```

```
}

// 4. KOMPOSIT TAHUN 2022
var median = filtered
  .map(cloudMask)
  .median()
  .clip(roi);

// 5. HITUNG NDBI DAN NDWI
var ndbi = median.expression(
  '(SWIR - NIR) / (SWIR + NIR)', {
    SWIR: median.select('B6'),
    NIR: median.select('B5')
  }).rename('NDBI');

var ndwi = median.expression(
  '(GREEN - NIR) / (GREEN + NIR)', {
    GREEN: median.select('B3'),
    NIR: median.select('B5')
  }).rename('NDWI');

// 6. CITRA UNTUK SAMPLING
var sampleImage = ndbi.addBands(ndwi);

// Buang piksel air berdasarkan kriteria NDWI <= 0.1
var nonWater = sampleImage.updateMask(ndwi.lte(0.1));

// Ambil sampel acak untuk mencari kandidat
var samples = nonWater.sample({
  region: roi,
  scale: 30,
  numPixels: 5000,
  seed: 42,
  geometries: true,
  dropNulls: true,
  tileScale: 4
});

print('Jumlah sampel valid:', samples.size());

// 7. PILIH 5 KANDIDAT DENGAN NDBI TERTINGGI
// Ini kandidat terbangun, perlu diperiksa secara visual.
var builtCandidates = samples
  .sort('NDBI', false)
  .limit(5)
  .map(function(f) {
```

```
    return f.set('kelas_kandidat', 'Terbangun');
  });

// 8. PILIH 5 KANDIDAT DENGAN NDBI TERENDAH
var nonBuiltCandidates = samples
  .sort('NDBI', true)
  .limit(5)
  .map(function(f) {
    return f.set('kelas_kandidat', 'Bukan terbangun');
  });

// 9. TAMPILKAN KOORDINAT DAN NILAI
print('5 kandidat terbangun:', builtCandidates);
print('5 kandidat bukan terbangun:', nonBuiltCandidates);

Map.centerObject(roi, 10);

Map.addLayer(median, {
  bands: ['B4', 'B3', 'B2'],
  min: 0.02,
  max: 0.30,
  gamma: 1.2
}, 'Landsat 8 RGB');

Map.addLayer(builtCandidates, {
  color: 'ff0000'
}, 'Kandidat terbangun');

Map.addLayer(nonBuiltCandidates, {
  color: '00ffff'
}, 'Kandidat bukan terbangun');

// 10. HITUNG RATA-RATA KEDUA KELOMPOK
var meanBuilt = ee.Number(
  builtCandidates.aggregate_mean('NDBI')
);

var meanNonBuilt = ee.Number(
  nonBuiltCandidates.aggregate_mean('NDBI')
);

// Ambang baru: titik tengah kedua rata-rata
var thresholdNew = meanBuilt
  .add(meanNonBuilt)
  .divide(2);
```

```
print('Rata-rata NDBI kandidat terbangun:', meanBuilt);
print('Rata-rata NDBI kandidat bukan terbangun:', meanNonBuilt);
print('Ambang awal:', -0.1);
print('Ambang baru eksperimental:', thresholdNew);

// 11. KLASIFIKASI DENGAN AMBANG LAMA DAN BARU
var waterMask = ndwi.lte(0.1);

var builtOld = ndbi.gt(-0.1)
  .and(waterMask)
  .rename('built_old');

var builtNew = ndbi.gt(thresholdNew)
  .and(waterMask)
  .rename('built_new');

// 12. HITUNG LUAS DALAM HEKTARE
function calculateArea(binaryImage) {
  var areaImage = ee.Image.pixelArea()
    .updateMask(binaryImage)
    .rename('area');

  var result = areaImage.reduceRegion({
    reducer: ee.Reducer.sum(),
    geometry: roi,
    scale: 30,
    maxPixels: 1e10,
    tileScale: 4
  });

  return ee.Number(
    ee.Algorithms.If(
      result.get('area'),
      result.get('area'),
      0
    )
  ).divide(10000);
}

var areaOld = calculateArea(builtOld);
var areaNew = calculateArea(builtNew);

var difference = areaNew.subtract(areaOld);

var percentChange = ee.Algorithms.If(
  areaOld.gt(0),
```

```
  difference.divide(areaOld).multiply(100),
  null
);

print('Luas ambang -0.1 (ha):', areaOld);
print('Luas ambang baru (ha):', areaNew);
print('Perubahan luas (ha):', difference);
print('Perubahan luas (%):', percentChange);

// 13. VISUALISASI PERBANDINGAN
Map.addLayer(
  builtOld.selfMask(),
  {palette: ['ff7070']},
  'Terbangun: ambang -0.1'
);

Map.addLayer(
  builtNew.selfMask(),
  {palette: ['00ffff']},
  'Terbangun: ambang baru'
);

// Kuning = piksel yang berubah kelas
var changed = builtOld.neq(builtNew);

Map.addLayer(
  changed.selfMask(),
  {palette: ['ffff00']},
  'Perubahan klasifikasi'
);
```

```
Jumlah citra sebelum masking:
44

Jumlah sampel valid:
4176

5 kandidat terbangun:
▸ FeatureCollection (5 elements, 3 columns)

5 kandidat bukan terbangun:
▸ FeatureCollection (5 elements, 3 columns)

Rata-rata NDBI kandidat terbangun:
0.2470044824325496

Rata-rata NDBI kandidat bukan terbangun:
-0.6229945707106748

Ambang awal:
-0.1

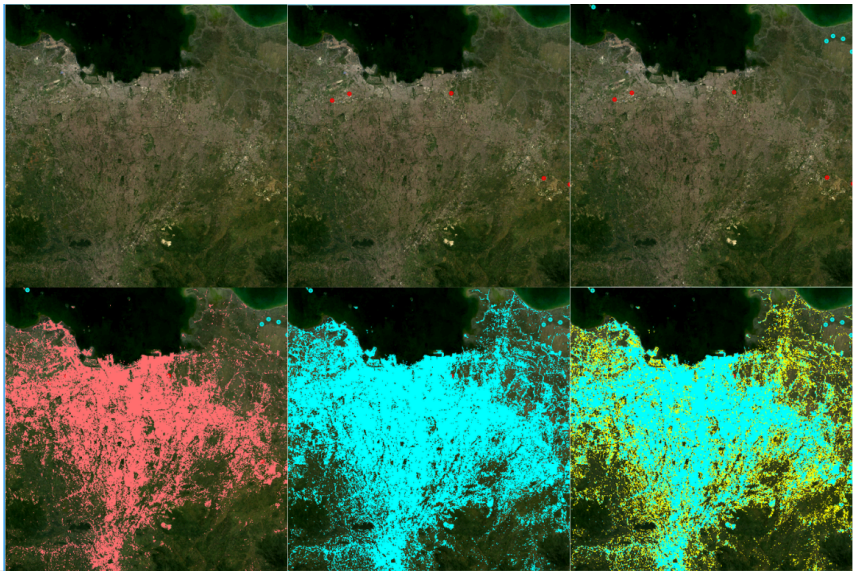
Ambang baru eksperimental:
-0.1879950441390599

Luas ambang -0.1 (ha):
175562.8868742355

Luas ambang baru (ha):
254742.388910846

Perubahan luas (ha):
79179.50203661047

Perubahan luas (%):
45.100364573824024
```



kalibrasi ambang batas NDBI (Normalized Difference Built-up Index) guna mengidentifikasi kandidat lahan terbangun pada wilayah kajian selama tahun 2022. Proses dimulai dengan mengambil citra satelit Landsat 8, menyaring citra berdasarkan wilayah dan periode waktu, menghilangkan piksel yang terkontaminasi awan, serta menggabungkan citra yang tersedia menjadi satu citra komposit. Selanjutnya, kode menghitung indeks NDBI untuk mengidentifikasi karakteristik lahan terbangun dan NDWI untuk membantu menyaring area yang berpotensi berupa air.

Setelah itu, kode mengambil sampel piksel yang bukan merupakan kandidat air, kemudian memilih lima sampel dengan nilai NDBI tertinggi sebagai kandidat lahan terbangun dan lima sampel dengan nilai NDBI terendah sebagai kandidat bukan lahan terbangun. Kedua kelompok kandidat tersebut ditampilkan pada peta untuk diperiksa secara visual. Berdasarkan rata-rata nilai NDBI dari kedua kelompok, kode menentukan ambang batas baru dengan mengambil nilai tengah kedua rata-rata tersebut. Ambang batas baru ini kemudian dibandingkan dengan ambang batas awal sebesar -0,1 untuk mengklasifikasikan piksel sebagai kandidat lahan terbangun.

Terakhir, kode menghitung dan membandingkan luas kandidat lahan terbangun yang dihasilkan oleh kedua ambang batas tersebut dalam satuan hektare, termasuk selisih luas dan persentase perubahannya. Hasil klasifikasi juga ditampilkan pada peta untuk memperlihatkan area yang teridentifikasi sebagai lahan terbangun berdasarkan ambang batas lama, ambang batas baru, serta area yang mengalami perubahan klasifikasi. Tujuan utamanya adalah mengevaluasi bagaimana penyesuaian ambang batas NDBI memengaruhi estimasi luas lahan terbangun, dengan catatan bahwa kandidat sampel yang dipilih tetap perlu divalidasi secara visual agar ambang baru yang diperoleh lebih dapat dipertanggungjawabkan.

3. Urban sprawl kota lain

```
/**********************************************************
 TASK III -- URBAN EXPANSION KOTA BANDUNG
 Tahun: 1990, 1995, 2000, 2005, 2010, 2015, 2020, 2024
**********************************************************/

// 1. ROI: KOTA BANDUNG
// Periksa jumlah fitur dan batas yang muncul di Console/Map.
var admin = ee.FeatureCollection('FAO/GAUL/2015/level2');

var bandungFC = admin
  .filter(ee.Filter.eq('ADM0_NAME', 'Indonesia'))
  .filter(ee.Filter.eq('ADM1_NAME', 'Jawa Barat'))
  .filter(ee.Filter.eq('ADM2_NAME', 'Bandung'));

print('Jumlah fitur bernama Bandung:', bandungFC.size());
print('Nama fitur yang ditemukan:',
bandungFC.aggregate_array('ADM2_NAME'));

var roi = bandungFC.geometry();

Map.centerObject(roi, 11);
```

```
Map.addLayer(
  bandungFC.style({color: 'FFFFFF', fillColor: '00000000', width: 2}),
  {},
  'Batas ROI'
);

// 2. KOLEKSI LANDSAT
var l4 = ee.ImageCollection('LANDSAT/LT04/C02/T1_L2');
var l5 = ee.ImageCollection('LANDSAT/LT05/C02/T1_L2');
var l7 = ee.ImageCollection('LANDSAT/LE07/C02/T1_L2');
var l8 = ee.ImageCollection('LANDSAT/LC08/C02/T1_L2');
var l9 = ee.ImageCollection('LANDSAT/LC09/C02/T1_L2');

var yearList = [1990, 1995, 2000, 2005, 2010, 2015, 2020, 2024];

function filterCol(col, geometry, start, end) {
  return col
    .filterBounds(geometry)
    .filterDate(start, end);
}

// 3. CLOUD MASK LANDSAT 4/5/7
function cloudMaskTm(image) {
  var qa = image.select('QA_PIXEL');

  var mask = qa.bitwiseAnd(1 << 1).eq(0)  // Dilated cloud
    .and(qa.bitwiseAnd(1 << 3).eq(0))     // Cloud
    .and(qa.bitwiseAnd(1 << 4).eq(0))     // Cloud shadow
    .and(qa.bitwiseAnd(1 << 5).eq(0));    // Snow

  return image.select(
    ['SR_B1', 'SR_B2', 'SR_B3',
     'SR_B4', 'SR_B5', 'SR_B7'],
    ['B2', 'B3', 'B4', 'B5', 'B6', 'B7']
  ).updateMask(mask)
   .copyProperties(image, ['system:time_start']);
}

// 4. CLOUD MASK LANDSAT 8/9
function cloudMaskOli(image) {
  var qa = image.select('QA_PIXEL');

  var mask = qa.bitwiseAnd(1 << 1).eq(0)
    .and(qa.bitwiseAnd(1 << 2).eq(0))     // Cirrus
    .and(qa.bitwiseAnd(1 << 3).eq(0))     // Cloud
    .and(qa.bitwiseAnd(1 << 4).eq(0))     // Cloud shadow
```

```
    .and(qa.bitwiseAnd(1 << 5).eq(0));    // Snow

  return image.select(
    ['SR_B2', 'SR_B3', 'SR_B4',
     'SR_B5', 'SR_B6', 'SR_B7'],
    ['B2', 'B3', 'B4', 'B5', 'B6', 'B7']
  ).updateMask(mask)
   .copyProperties(image, ['system:time_start']);
}

// 5. KOMPOSIT MEDIAN
function landsat457(geometry, start, end) {
  var col = filterCol(l4, geometry, start, end)
    .merge(filterCol(l5, geometry, start, end))
    .merge(filterCol(l7, geometry, start, end));

  return col.map(cloudMaskTm).median().clip(geometry);
}

function landsat89(geometry, start, end) {
  var col = filterCol(l8, geometry, start, end)
    .merge(filterCol(l9, geometry, start, end));

  return col.map(cloudMaskOli).median().clip(geometry);
}

// 6. KLASIFIKASI DAN LUAS TAHUNAN
var builtImages = yearList.map(function(year) {
  var start = ee.Date.fromYMD(year, 1, 1);
  var end = ee.Date.fromYMD(year + 1, 1, 1);

  var raw = (year < 2014)
    ? landsat457(roi, start, end)
    : landsat89(roi, start, end);

  // Skalakan reflektansi Collection 2
  var image = raw.multiply(0.0000275).add(-0.2);

  // RGB false color untuk inspeksi
  Map.addLayer(
    image,
    {
      bands: ['B5', 'B6', 'B2'],
      min: 0.0,
      max: 0.4,
      gamma: 1.2
```

```
  },
  'Komposit_' + year,
  false
);

// NDBI = (SWIR1 - NIR) / (SWIR1 + NIR)
var ndbi = image.expression(
  '(SWIR - NIR) / (SWIR + NIR)', {
    SWIR: image.select('B6'),
    NIR: image.select('B5')
  }).rename('NDBI');

// MNDWI = (Green - SWIR1) / (Green + SWIR1)
var mndwi = image.expression(
  '(GREEN - SWIR) / (GREEN + SWIR)', {
    GREEN: image.select('B3'),
    SWIR: image.select('B6')
  }).rename('MNDWI');

// Ambang eksploratif yang dipertahankan dari kode awal
var builtMask = ndbi.gt(-0.1)
  .and(mndwi.lte(0))
  .rename('built_mask');

// Simpan nilai tahun hanya di piksel terbangun
var builtYear = ee.Image.constant(year)
  .toUint16()
  .rename('built')
  .updateMask(builtMask)
  .clip(roi);

// Luas piksel terbangun dalam hektare
var areaHa = ee.Image.pixelArea()
  .divide(10000)
  .rename('area_ha')
  .updateMask(builtMask)
  .clip(roi);

Map.addLayer(
  builtYear,
  {min: year, max: year, palette: ['FF0000']},
  'Terbangun_' + year,
  false
);

return builtYear
```

```
    .addBands(areaHa)
    .set('year', year)
    .set('system:time_start', start.millis());
});

var builtCol = ee.ImageCollection.fromImages(builtImages);

// 7. PETA TAHUN PERTAMA TERDETEKSINYA LAHAN TERBANGUN
// min() mengambil tahun paling awal dari piksel yang terdeteksi
// sebagai terbangun pada salah satu tahun pengamatan.
var urbanExpansion = builtCol.select('built').min();

var palette = [
  '800080', // 1990
  '0000FF', // 1995
  '00FFFF', // 2000
  '008000', // 2005
  'FFFF00', // 2010
  'FFA500', // 2015
  'FF0000', // 2020
  'FFFFFF'  // 2024
];

// Ubah tahun menjadi kelas indeks 0-7 untuk legenda kategoris
var classValues = [0, 1, 2, 3, 4, 5, 6, 7];

var expansionClass = urbanExpansion
  .remap(yearList, classValues)
  .rename('first_detection_class');

Map.addLayer(
  expansionClass,
  {
    min: 0,
    max: 7,
    palette: palette
  },
  'Urban expansion -- tahun deteksi pertama',
  true
);

// 8. HITUNG LUAS TERBANGUN PER TAHUN
var areaFeatures = yearList.map(function(year) {
  var annualImage = ee.Image(
    builtCol.filter(ee.Filter.eq('year', year)).first()
  );
```

```
  var result = annualImage.select('area_ha').reduceRegion({
    reducer: ee.Reducer.sum(),
    geometry: roi,
    scale: 30,
    maxPixels: 1e10,
    tileScale: 4
  });

  var area = ee.Number(
    ee.Algorithms.If(
      result.contains('area_ha'),
      result.get('area_ha'),
      0
    )
  );

  return ee.Feature(null, {
    year: year,
    area_ha: area
  });
});

var areaFC = ee.FeatureCollection(areaFeatures);

print('Tabel luas terbangun tahunan (ha):', areaFC);

// 9. GRAFIK LUAS TERBANGUN
var chart = ui.Chart.feature.byFeature(
  areaFC.sort('year'),
  'year',
  ['area_ha']
)
.setChartType('LineChart')
.setOptions({
  title: 'Perubahan Luas Lahan Terbangun -- Kota Bandung',
  hAxis: {
    title: 'Tahun',
    format: '####',
    gridlines: {count: 8}
  },
  vAxis: {
    title: 'Luas lahan terbangun (hektare)',
    minValue: 0
  },
  lineWidth: 2,
```

```
  pointSize: 5,
  legend: {position: 'none'}
});

print(chart);

// 10. LEGENDA PETA EKSPANSI
var legend = ui.Panel({
  style: {
    position: 'bottom-left',
    padding: '8px 12px'
  }
});

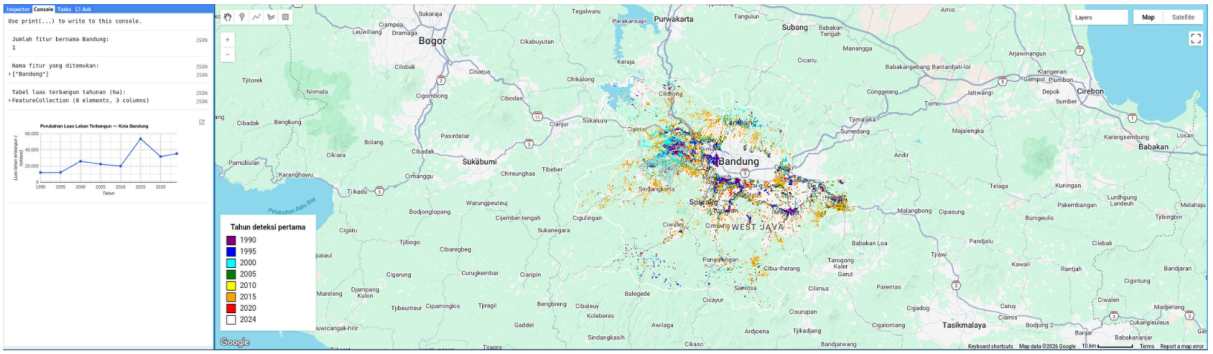
legend.add(ui.Label({
  value: 'Tahun deteksi pertama',
  style: {fontWeight: 'bold', fontSize: '14px'}
}));

yearList.forEach(function(year, index) {
  var colorBox = ui.Label('', {
    backgroundColor: '#' + palette[index],
    padding: '8px',
    margin: '0 8px 4px 0',
    border: '1px solid black'
  });

  var label = ui.Label(String(year), {
    margin: '0 0 4px 0'
  });

  legend.add(ui.Panel(
    [colorBox, label],
    ui.Panel.Layout.flow('horizontal')
  ));
});

Map.add(legend);
```



Kode ini bertujuan untuk menganalisis perubahan dan ekspansi lahan terbangun di Kota Bandung dari tahun 1990 hingga 2024 menggunakan citra satelit Landsat. Analisis dilakukan pada delapan tahun pengamatan, yaitu 1990, 1995, 2000, 2005, 2010, 2015, 2020, dan 2024. Kode menentukan batas wilayah kajian berdasarkan data administrasi, mengambil citra Landsat yang sesuai dengan setiap periode, menghilangkan piksel yang terkontaminasi awan, serta menggabungkan citra menjadi komposit tahunan untuk menggambarkan kondisi permukaan wilayah pada masing-masing tahun.

Selanjutnya, kode menggunakan indeks NDBI dan MNDWI untuk mengidentifikasi kandidat lahan terbangun sekaligus membantu mengecualikan area perairan. Identifikasi dilakukan menggunakan ambang batas NDBI lebih besar dari -0,1 dan MNDWI kurang dari atau sama dengan 0. Hasil klasifikasi setiap tahun kemudian digunakan untuk membuat peta ekspansi perkotaan yang menunjukkan tahun paling awal suatu piksel terdeteksi sebagai lahan terbangun selama periode pengamatan. Dengan demikian, peta tersebut membantu memperlihatkan persebaran spasial kawasan yang teridentifikasi sebagai lahan terbangun sejak awal hingga akhir periode penelitian.

Terakhir, kode menghitung luas kandidat lahan terbangun pada setiap tahun dalam satuan hektare dan menyajikan hasilnya dalam bentuk tabel serta grafik garis. Hasil tersebut digunakan untuk membandingkan perubahan luas lahan terbangun dari waktu ke waktu dan mengidentifikasi periode ketika luasnya mengalami peningkatan atau penurunan. Peta ekspansi juga dapat membantu mengamati pola perkembangan kawasan perkotaan, baik di sekitar kawasan terbangun sebelumnya maupun di wilayah pinggiran. Namun, hasilnya merupakan estimasi berdasarkan ambang indeks yang digunakan, sehingga perubahan yang terdeteksi perlu divalidasi untuk memastikan bahwa perubahan tersebut benar-benar mencerminkan ekspansi lahan terbangun, bukan variasi kondisi permukaan atau perbedaan kualitas citra antarperiode.

4. Risiko tiga komponen

```
function cloudMask(image) {
  var qa = image.select('QA_PIXEL');      // band penanda kualitas piksel
  var dilated = 1 << 1;                   // bit 1: awan melebar
  var cirrus  = 1 << 2;                   // bit 2: awan cirrus (tipis)
  var cloud   = 1 << 3;                   // bit 3: awan
  var shadow  = 1 << 4;                   // bit 4: bayangan awan
  var mask = qa.bitwiseAnd(dilated).eq(0)
    .and(qa.bitwiseAnd(cirrus).eq(0))
```

```
    .and(qa.bitwiseAnd(cloud).eq(0))
    .and(qa.bitwiseAnd(shadow).eq(0));
  var rename = image.select(
    ['SR_B2', 'SR_B3', 'SR_B4', 'SR_B5', 'SR_B6', 'SR_B7'],
    ['B2', 'B3', 'B4', 'B5', 'B6', 'B7']);
  var scaled = rename.multiply(0.0000275).add(-0.2);
  return scaled.updateMask(mask);
}


var roi = ee.Geometry({
  geodesic: false,
  type: 'Polygon',
  coordinates: [[
    [103.47443690212873, -1.7079691352593016],
    [103.7202559939256, -1.7079691352593016],
    [103.7202559939256, -1.469108505971908],
    [103.47443690212873, -1.469108505971908],
    [103.47443690212873, -1.7079691352593016]
  ]]
});

var filtered = ee.ImageCollection('LANDSAT/LC08/C02/T1_L2')
  .filterBounds(roi)
  .filterDate('2022-01-01', '2022-12-31');

// Fungsi cloudMask identik Task II -- salin ke skrip ini
var image = filtered.map(cloudMask).median().clip(roi);

Map.addLayer(image, {bands: ['B5', 'B6', 'B2'], min: [0.1, 0.05,
0], max: [0.4, 0.3, 0.15]}, 'Image');
Map.centerObject(roi, 11);

var bandMap = {
  NIR: image.select('B5'),
  SWIR: image.select('B6'),
  GREEN: image.select('B3'),
  RED: image.select('B4'),
  BLUE: image.select('B2')
};
```

```
// Badan air
var ndwi = image.expression('(GREEN - NIR) / (GREEN + NIR)',
bandMap);
var air = ndwi.gt(0.1);
var bukanAir = air.eq(0);
Map.addLayer(air, {palette: ['black', 'blue'], min: 0, max: 1},
'Air', false);

var srtm =
ee.Image('USGS/SRTMGL1_003').clip(roi).updateMask(bukanAir);
Map.addLayer(srtm, {min: 0, max: 200, palette: ['green', 'yellow',
'red']}, 'Elevasi', false);

var harkatElevasi = srtm.where(srtm.gt(20), 1)
  .where(srtm.gt(15).and(srtm.lte(20)), 2)
  .where(srtm.gt(10).and(srtm.lte(15)), 3)
  .where(srtm.gt(5).and(srtm.lte(10)), 4)
  .where(srtm.lte(5), 5);
Map.addLayer(harkatElevasi, {min: 1, max: 5, palette: ['green',
'yellow', 'red']}, 'Harkat elevasi', false);

var tpi = srtm.subtract(srtm.focalMean(3))
  .reproject('EPSG:4326', null, 30)
  .updateMask(bukanAir)
  .clip(roi);
Map.addLayer(tpi, {min: -10, max: 10, palette: ['blue', 'yellow',
'red']}, 'TPI', false);

var harkatTPI = tpi.where(tpi.gt(0), 1)
  .where(tpi.gt(-0.25).and(tpi.lte(0)), 2)
  .where(tpi.gt(-0.5).and(tpi.lte(-0.25)), 3)
  .where(tpi.gt(-0.75).and(tpi.lte(-0.5)), 4)
  .where(tpi.lte(-0.75), 5);
Map.addLayer(harkatTPI, {min: 1, max: 5, palette: ['green',
'yellow', 'red']}, 'Harkat TPI', false);

var jarakAir = air.fastDistanceTransform()
  .sqrt().multiply(30)      // jarak kuadrat-piksel -> meter
  .updateMask(bukanAir)
  .clip(roi)
  .reproject('EPSG:4326', null, 30);
Map.addLayer(jarakAir, {min: 0, max: 1000, palette: ['red',
```

```
'yellow', 'green']}, 'Jarak dari air', false);

// Harkat jarak (Tabel 4)
var harkatJarakAir = jarakAir.where(jarakAir.gt(200), 1)
  .where(jarakAir.gt(150).and(jarakAir.lte(200)), 2)
  .where(jarakAir.gt(100).and(jarakAir.lte(150)), 3)
  .where(jarakAir.gt(50).and(jarakAir.lte(100)), 4)
  .where(jarakAir.lte(50), 5);
Map.addLayer(harkatJarakAir, {min: 1, max: 5, palette: ['green',
'yellow', 'red']}, 'Harkat jarak dari air', false);

var ndvi = image.expression('(NIR - RED) / (NIR + RED)',
bandMap).updateMask(bukanAir);
Map.addLayer(ndvi, {min: -1, max: 1, palette: ['blue', 'white',
'green']}, 'NDVI', false);

var harkatNDVI = ndvi.where(ndvi.gt(0.5), 1)
  .where(ndvi.gt(0.25).and(ndvi.lte(0.5)), 2)
  .where(ndvi.gt(0).and(ndvi.lte(0.25)), 3)
  .where(ndvi.gt(-0.25).and(ndvi.lte(0)), 4)
  .where(ndvi.lte(-0.25), 5);
Map.addLayer(harkatNDVI, {min: 1, max: 5, palette: ['green',
'yellow', 'red']}, 'Harkat NDVI', false);

var kerawanan =
harkatJarakAir.add(harkatElevasi).add(harkatTPI).add(harkatNDVI);
Map.addLayer(kerawanan, {min: 1, max: 20, palette: ['green',
'yellow', 'red']}, 'Kerawanan', false);

var harkatKerawanan = kerawanan.where(kerawanan.gt(16), 5)
  .where(kerawanan.gt(12).and(kerawanan.lte(16)), 4)
  .where(kerawanan.gt(8).and(kerawanan.lte(12)), 3)
  .where(kerawanan.gt(4).and(kerawanan.lte(8)), 2)
  .where(kerawanan.lte(4), 1);
Map.addLayer(harkatKerawanan, {min: 1, max: 5, palette: ['green',
'yellow', 'red']}, 'Harkat kerawanan', false);

var penduduk = ee.ImageCollection('JRC/GHSL/P2023A/GHS_POP')
  .sort('system:time_start', false)
  .first().updateMask(bukanAir).clip(roi);
Map.addLayer(penduduk, {min: 0, max: 250, palette: ['black',
'red', 'white']}, 'Penduduk', false);
```

```
var harkatPenduduk = penduduk.where(penduduk.gt(300), 5)
  .where(penduduk.gt(200).and(penduduk.lte(300)), 4)
  .where(penduduk.gt(100).and(penduduk.lte(200)), 3)
  .where(penduduk.gt(10).and(penduduk.lte(100)), 2)
  .where(penduduk.lte(10), 1);
Map.addLayer(harkatPenduduk, {min: 1, max: 5, palette: ['green',
'yellow', 'red']}, 'Harkat penduduk', false);


var wc =
ee.ImageCollection('ESA/WorldCover/v200').first().clip(roi).update
Mask(bukanAir);

var lahanTerbangun = wc.eq(50);   // kelas 50 = built-up
var pertanian = wc.eq(40);        // kelas 40 = cropland

var area = ee.Image.pixelArea();

// Luas terbangun per piksel 30 m (m persegi)
var lahanTerbangunArea = lahanTerbangun.multiply(area)
  .reduceResolution(ee.Reducer.sum(), true)
  .reproject('EPSG:4326', null, 30);
Map.addLayer(lahanTerbangunArea, {min: 0, max: 100, palette:
['black', 'red', 'white']}, 'Luas lahan terbangun', false);

var harkatLahanTerbangun =
lahanTerbangunArea.where(lahanTerbangunArea.gt(100), 5)

.where(lahanTerbangunArea.gt(80).and(lahanTerbangunArea.lte(100)),
4)

.where(lahanTerbangunArea.gt(60).and(lahanTerbangunArea.lte(80)),
3)

.where(lahanTerbangunArea.gt(40).and(lahanTerbangunArea.lte(60)),
2)
  .where(lahanTerbangunArea.lte(40), 1);
Map.addLayer(harkatLahanTerbangun, {min: 1, max: 5, palette:
['green', 'yellow', 'red']}, 'Harkat lahan terbangun', false);

// Luas pertanian per piksel 30 m + harkat
```

```
var pertanianArea = pertanian.multiply(area)
  .reduceResolution(ee.Reducer.sum(), true)
  .reproject('EPSG:4326', null, 30);
Map.addLayer(pertanianArea, {min: 0, max: 100, palette: ['black',
'lightgreen', 'darkgreen']}, 'Luas pertanian', false);

var harkatPertanian = pertanianArea.where(pertanianArea.gt(100),
5)
  .where(pertanianArea.gt(80).and(pertanianArea.lte(100)), 4)
  .where(pertanianArea.gt(60).and(pertanianArea.lte(80)), 3)
  .where(pertanianArea.gt(40).and(pertanianArea.lte(60)), 2)
  .where(pertanianArea.lte(40), 1);
Map.addLayer(harkatPertanian, {min: 1, max: 5, palette: ['green',
'yellow', 'red']}, 'Harkat pertanian', false);


var eksposur =
harkatPenduduk.add(harkatLahanTerbangun).add(harkatPertanian);
Map.addLayer(eksposur, {min: 1, max: 15, palette: ['green',
'yellow', 'red']}, 'Eksposur', false);

var harkatEksposur = eksposur.where(eksposur.gt(12), 5)
  .where(eksposur.gt(9).and(eksposur.lte(12)), 4)
  .where(eksposur.gt(6).and(eksposur.lte(9)), 3)
  .where(eksposur.gt(3).and(eksposur.lte(6)), 2)
  .where(eksposur.lte(3), 1);
Map.addLayer(harkatEksposur, {min: 1, max: 5, palette: ['green',
'yellow', 'red']}, 'Harkat eksposur', false);


// PDB per piksel (USD)
var pdb =
ee.Image('projects/sat-io/open-datasets/GRIDDED_HDI_GDP/GDP_PPP_19
90_2015_5arcmin_v2')
  .select('b26').clip(roi).updateMask(bukanAir);
Map.addLayer(pdb, {min: 0, max: 1e8, palette: ['black', 'green',
'white']}, 'PDB', false);

var harkatPDB = pdb.where(pdb.gt(40 * 1e6), 1)
  .where(pdb.gt(30 * 1e6).and(pdb.lte(40 * 1e6)), 2)
  .where(pdb.gt(20 * 1e6).and(pdb.lte(30 * 1e6)), 3)
  .where(pdb.gt(10 * 1e6).and(pdb.lte(20 * 1e6)), 4)
```

```
  .where(pdb.lte(10 * 1e6), 5);
Map.addLayer(harkatPDB, {min: 1, max: 5, palette: ['green',
'yellow', 'red']}, 'Harkat PDB', false);

// IPM (0--1)
var ipm =
ee.Image('projects/sat-io/open-datasets/GRIDDED_HDI_GDP/HDI_1990_2
015_v2')
  .select('b26').clip(roi).updateMask(bukanAir);
Map.addLayer(ipm, {min: 0.5, max: 1, palette: ['black', 'blue',
'white']}, 'IPM', false);

var harkatIPM = ipm.where(ipm.gt(0.85), 1)
  .where(ipm.gt(0.8).and(ipm.lte(0.85)), 2)
  .where(ipm.gt(0.75).and(ipm.lte(0.8)), 3)
  .where(ipm.gt(0.7).and(ipm.lte(0.75)), 4)
  .where(ipm.lte(0.7), 5);
Map.addLayer(harkatIPM, {min: 1, max: 5, palette: ['green',
'yellow', 'red']}, 'Harkat IPM', false);

var kerentanan = harkatPDB.add(harkatIPM);
Map.addLayer(kerentanan, {min: 1, max: 10, palette: ['green',
'yellow', 'red']}, 'Kerentanan', false);

var harkatKerentanan = kerentanan.where(kerentanan.gt(8), 5)
  .where(kerentanan.gt(6).and(kerentanan.lte(8)), 4)
  .where(kerentanan.gt(4).and(kerentanan.lte(6)), 3)
  .where(kerentanan.gt(2).and(kerentanan.lte(4)), 2)
  .where(kerentanan.lte(2), 1);
Map.addLayer(harkatKerentanan, {min: 1, max: 5, palette: ['green',
'yellow', 'red']}, 'Harkat kerentanan', false);


var risiko = harkatKerawanan.add(harkatEksposur);   //
.add(harkatKerentanan);
Map.addLayer(risiko, {min: 1, max: 10, palette: ['green',
'yellow', 'red']}, 'Risiko banjir', false);

var harkatRisiko = risiko.where(risiko.gt(8), 5)
  .where(risiko.gt(6).and(risiko.lte(8)), 4)
  .where(risiko.gt(4).and(risiko.lte(6)), 3)
  .where(risiko.gt(2).and(risiko.lte(4)), 2)
```

```
  .where(risiko.lte(2), 1);
Map.addLayer(harkatRisiko, {min: 1, max: 5, palette: ['green',
'yellow', 'red']}, 'Harkat risiko', false);


/*************************************************************
 * PERBANDINGAN RISIKO BANJIR SEBELUM DAN SESUDAH KERENTANAN
 *************************************************************/

// ===========================================================
// A. RISIKO SEBELUM: KERAWANAN + EKSPOSUR
// ===========================================================

var risikoSebelum = harkatKerawanan
  .add(harkatEksposur)
  .rename('risiko_sebelum');

var harkatRisikoSebelum = risikoSebelum
  .where(risikoSebelum.gt(8), 5)
  .where(risikoSebelum.gt(6).and(risikoSebelum.lte(8)), 4)
  .where(risikoSebelum.gt(4).and(risikoSebelum.lte(6)), 3)
  .where(risikoSebelum.gt(2).and(risikoSebelum.lte(4)), 2)
  .where(risikoSebelum.lte(2), 1)
  .rename('kelas_sebelum');

Map.addLayer(
  harkatRisikoSebelum,
  {min: 1, max: 5,
   palette: ['006400', '7FFF00', 'FFFF00', 'FFA500', 'FF0000']},
  'Harkat risiko sebelum kerentanan',
  true
);


// ===========================================================
// B. RISIKO SESUDAH: KERAWANAN + EKSPOSUR + KERENTANAN
// ===========================================================

var risikoSesudah = harkatKerawanan
  .add(harkatEksposur)
  .add(harkatKerentanan)
  .rename('risiko_sesudah');
```

```
var harkatRisikoSesudah = risikoSesudah
  .where(risikoSesudah.gt(8), 5)
  .where(risikoSesudah.gt(6).and(risikoSesudah.lte(8)), 4)
  .where(risikoSesudah.gt(4).and(risikoSesudah.lte(6)), 3)
  .where(risikoSesudah.gt(2).and(risikoSesudah.lte(4)), 2)
  .where(risikoSesudah.lte(2), 1)
  .rename('kelas_sesudah');

Map.addLayer(
  harkatRisikoSesudah,
  {min: 1, max: 5,
   palette: ['006400', '7FFF00', 'FFFF00', 'FFA500', 'FF0000']},
  'Harkat risiko sesudah kerentanan',
  false
);


// ============================================================
// C. PETA PERUBAHAN KELAS RISIKO
// ============================================================

// Nilai positif: kelas meningkat
// Nilai nol: kelas tidak berubah
// Nilai negatif: kelas menurun

var perubahanKelas = harkatRisikoSesudah
  .subtract(harkatRisikoSebelum)
  .rename('perubahan_kelas');

Map.addLayer(
  perubahanKelas,
  {min: -4, max: 4,
   palette: ['0000FF', 'FFFFFF', 'FF0000']},
  'Perubahan kelas risiko',
  false
);


// ============================================================
// D. FUNGSI MENGHITUNG LUAS KELAS RISIKO (HEKTARE)
// Tanpa Reducer.group dan List.filter
```

```
// ============================================================

var luasPikselHa = ee.Image.pixelArea()
  .divide(10000)
  .rename('luas_ha');

function hitungLuasKelas(image, nama) {

  var tabel = ee.FeatureCollection(
    ee.List.sequence(1, 5).map(function(kelas) {

      // Ambil hanya piksel yang memiliki kelas tersebut
      var maskKelas = image.eq(ee.Number(kelas));

      var hasil = luasPikselHa
        .updateMask(maskKelas)
        .reduceRegion({
          reducer: ee.Reducer.sum(),
          geometry: roi,
          scale: 30,
          crs: 'EPSG:32748',
          maxPixels: 1e13,
          tileScale: 4
        });

      var luas = ee.Number(
        ee.Algorithms.If(
          hasil.contains('luas_ha'),
          hasil.get('luas_ha'),
          0
        )
      );

      return ee.Feature(null, {
        skenario: nama,
        kelas_risiko: kelas,
        luas_ha: luas
      });
    })
  );

  return tabel;
```

```
}


// ============================================================
// E. HITUNG LUAS SEBELUM DAN SESUDAH
// ============================================================

var luasSebelum = hitungLuasKelas(
  harkatRisikoSebelum,
  'Sebelum'
);

var luasSesudah = hitungLuasKelas(
  harkatRisikoSesudah,
  'Sesudah'
);

print(
  'Luas tiap kelas sebelum (ha)',
  luasSebelum
);

print(
  'Luas tiap kelas sesudah (ha)',
  luasSesudah
);


// ============================================================
// F. TABEL PERBANDINGAN LUAS
// ============================================================

var perbandinganLuas = ee.FeatureCollection(
  ee.List.sequence(1, 5).map(function(kelas) {

    var sebelum = ee.Number(
      luasSebelum
        .filter(ee.Filter.eq('kelas_risiko', kelas))
        .first()
        .get('luas_ha')
    );
```

```
    var sesudah = ee.Number(
      luasSesudah
        .filter(ee.Filter.eq('kelas_risiko', kelas))
        .first()
        .get('luas_ha')
    );

    var perubahan = sesudah.subtract(sebelum);

    var persen = ee.Algorithms.If(
      sebelum.gt(0),
      perubahan.divide(sebelum).multiply(100),
      null
    );

    return ee.Feature(null, {
      kelas_risiko: kelas,
      sebelum_ha: sebelum,
      sesudah_ha: sesudah,
      perubahan_ha: perubahan,
      perubahan_persen: persen
    });
  })
);

print(
  'Perbandingan luas kelas risiko',
  perbandinganLuas
);


// ============================================================
// G. MATRIKS PERPINDAHAN KELAS RISIKO (HEKTARE)
// Baris = kelas sebelum
// Kolom = kelas sesudah
// ============================================================

var matriksPerubahan = ee.FeatureCollection(
  ee.List.sequence(1, 5).map(function(kelasAwal) {

    var fiturBaris = ee.FeatureCollection(
      ee.List.sequence(1, 5).map(function(kelasAkhir) {
```

```
        var maskTransisi = harkatRisikoSebelum
          .eq(ee.Number(kelasAwal))
          .and(
            harkatRisikoSesudah.eq(ee.Number(kelasAkhir))
          );

        var hasil = luasPikselHa
          .updateMask(maskTransisi)
          .reduceRegion({
            reducer: ee.Reducer.sum(),
            geometry: roi,
            scale: 30,
            crs: 'EPSG:32748',
            maxPixels: 1e13,
            tileScale: 4
          });

        var luas = ee.Number(
          ee.Algorithms.If(
            hasil.contains('luas_ha'),
            hasil.get('luas_ha'),
            0
          )
        );

        return ee.Feature(null, {
          kelas_sebelum: kelasAwal,
          kelas_sesudah: kelasAkhir,
          luas_ha: luas
        });
      })
    );

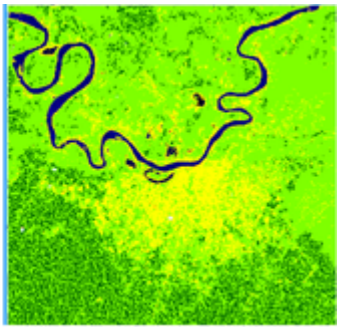
    return fiturBaris;
  })
).flatten();

print(
  'Matriks perpindahan kelas risiko (ha)',
  matriksPerubahan
);
```

```
Luas tiap kelas sebelum (ha)                                    JSON
▸FeatureCollection (5 elements, 4 columns)                      JSON

Luas tiap kelas sesudah (ha)                                    JSON
▸FeatureCollection (5 elements, 4 columns)                      JSON

Perbandingan luas kelas risiko                                  JSON
▸FeatureCollection (5 elements, 6 columns)                      JSON
```



Kode ini digunakan untuk menganalisis risiko banjir di wilayah kajian berdasarkan faktor fisik lingkungan dan kondisi sosial ekonomi menggunakan Google Earth Engine. Analisis dimulai dengan mengolah citra Landsat 8 untuk mengidentifikasi badan air dan menyaring area perairan dari perhitungan berikutnya. Selanjutnya, kode memanfaatkan data elevasi untuk menilai ketinggian dan karakteristik topografi, menghitung jarak dari badan air, serta menggunakan indeks vegetasi untuk menggambarkan kondisi tutupan vegetasi. Setiap faktor tersebut kemudian diberi nilai tingkat atau harkat untuk membantu menyusun peta kerawanan banjir.

Selain kerawanan, kode juga menganalisis eksposur dan kerentanan wilayah. Eksposur dinilai berdasarkan kepadatan penduduk, luas lahan terbangun, dan luas lahan pertanian, sedangkan kerentanan dinilai menggunakan data Produk Domestik Bruto (PDB) dan Indeks Pembangunan Manusia (IPM). Setiap variabel dikelompokkan ke dalam beberapa tingkat harkat, kemudian digabungkan untuk menghasilkan peta eksposur dan kerentanan. Hasil pengolahan ini digunakan untuk membentuk peta risiko banjir yang menggambarkan tingkat risiko relatif di setiap lokasi berdasarkan kombinasi faktor-faktor yang dianalisis.

Terakhir, kode membandingkan risiko banjir sebelum dan sesudah faktor kerentanan diperhitungkan. Kode menghasilkan peta perubahan kelas risiko, menghitung luas setiap kelas risiko dalam hektare, menyajikan tabel perbandingan luas beserta perubahan persentasenya, dan membuat matriks perpindahan kelas untuk menunjukkan perubahan tingkat risiko di antara kedua skenario. Hasilnya dapat digunakan untuk melihat bagaimana penambahan faktor sosial ekonomi mengubah distribusi spasial risiko banjir. Namun, peta ini merupakan hasil pemodelan berdasarkan variabel dan pembobotan yang ditetapkan, sehingga interpretasinya perlu mempertimbangkan kesesuaian data, asumsi klasifikasi, dan validasi terhadap kondisi banjir nyata.

Tugas HOTS

1. Sensitivitas clustering

```
/**********************************************************
 PERBANDINGAN MASKING AWAN LANDSAT 8 -- 2022
 ROI: area Jabodetabekpunjur berdasarkan koordinat script
**********************************************************/

// 1. ROI
var roi = ee.Geometry({
  geodesic: false,
  type: 'Polygon',
  coordinates: [[
    [106.53598136542733, -6.6221756519036195],
    [107.23910636542733, -6.6221756519036195],
    [107.23910636542733, -5.928728445061094],
    [106.53598136542733, -5.928728445061094],
    [106.53598136542733, -6.6221756519036195]
  ]]
});

// 2. KOLEKSI LANDSAT 8 TAHUN 2022
var l8 = ee.ImageCollection('LANDSAT/LC08/C02/T1_L2');

var filtered = l8
  .filterBounds(roi)
  .filterDate('2022-01-01', '2023-01-01');

print('Jumlah citra Landsat 8:', filtered.size());

// 3. MASKING KETAT -- METODE MODUL
// Piksel ditolak jika minimal satu dari empat bit menyala.
function cloudMask(image) {
  var qa = image.select('QA_PIXEL');

  var dilated = 1 << 1;
  var cirrus  = 1 << 2;
  var cloud   = 1 << 3;
  var shadow  = 1 << 4;

  var mask = qa.bitwiseAnd(dilated).eq(0)
    .and(qa.bitwiseAnd(cirrus).eq(0))
    .and(qa.bitwiseAnd(cloud).eq(0))
    .and(qa.bitwiseAnd(shadow).eq(0));

  var scaled = image.select(
    ['SR_B2', 'SR_B3', 'SR_B4',
```

```
      'SR_B5', 'SR_B6', 'SR_B7'],
    ['B2', 'B3', 'B4', 'B5', 'B6', 'B7']
  ).multiply(0.0000275).add(-0.2);

  return scaled.updateMask(mask)
    .copyProperties(image, ['system:time_start']);
}

// 4. MASKING KUSTOM
// Piksel ditolak hanya jika minimal DUA bit menyala.
function maskAwanKustom(image) {
  var qa = image.select('QA_PIXEL');

  var dilated = qa.bitwiseAnd(1 << 1).gt(0);
  var cirrus  = qa.bitwiseAnd(1 << 2).gt(0);
  var cloud   = qa.bitwiseAnd(1 << 3).gt(0);
  var shadow  = qa.bitwiseAnd(1 << 4).gt(0);

  // Konversi Boolean ke bilangan agar dapat dijumlahkan.
  var jumlahBit = dilated.toInt()
    .add(cirrus.toInt())
    .add(cloud.toInt())
    .add(shadow.toInt());

  // Pertahankan piksel jika jumlah bit aktif kurang dari 2.
  var mask = jumlahBit.lt(2);

  var scaled = image.select(
    ['SR_B2', 'SR_B3', 'SR_B4',
     'SR_B5', 'SR_B6', 'SR_B7'],
    ['B2', 'B3', 'B4', 'B5', 'B6', 'B7']
  ).multiply(0.0000275).add(-0.2);

  return scaled.updateMask(mask)
    .copyProperties(image, ['system:time_start']);
}

// 5. KOMPOSIT MEDIAN UNTUK KEDUA METODE
var medianKetat = filtered
  .map(cloudMask)
  .median()
  .clip(roi);

var medianKustom = filtered
  .map(maskAwanKustom)
  .median()
```

```
  .clip(roi);

// 6. HITUNG JUMLAH PIXEL VALID PADA KOMPOSIT
// Menghitung lokasi piksel valid, bukan jumlah observasi
// dari seluruh citra dalam koleksi.
function hitungPikselValid(image, namaBand) {
  var hasil = image.select(namaBand).reduceRegion({
    reducer: ee.Reducer.count(),
    geometry: roi,
    scale: 30,
    maxPixels: 1e10,
    tileScale: 4
  });

  return ee.Number(hasil.get(namaBand));
}

var jumlahKetat = hitungPikselValid(medianKetat, 'B5');
var jumlahKustom = hitungPikselValid(medianKustom, 'B5');

var selisihPiksel = jumlahKustom.subtract(jumlahKetat);

var persenTambahan = ee.Algorithms.If(
  jumlahKetat.gt(0),
  selisihPiksel.divide(jumlahKetat).multiply(100),
  null
);

print('Piksel valid -- masking ketat:', jumlahKetat);
print('Piksel valid -- masking kustom:', jumlahKustom);
print('Tambahan piksel valid:', selisihPiksel);
print('Tambahan piksel valid (%):', persenTambahan);

// 7. VISUALISASI KOMPOSIT
var rgb = {
  bands: ['B4', 'B3', 'B2'],
  min: 0.02,
  max: 0.30,
  gamma: 1.2
};

Map.centerObject(roi, 10);

Map.addLayer(medianKetat, rgb, 'RGB -- masking ketat', true);
Map.addLayer(medianKustom, rgb, 'RGB -- masking kustom', false);
```

```
// 8. HITUNG NDBI DAN NDWI
function hitungIndeks(image) {
  var ndbi = image.expression(
    '(SWIR - NIR) / (SWIR + NIR)', {
      SWIR: image.select('B6'),
      NIR: image.select('B5')
    }).rename('NDBI');

  var ndwi = image.expression(
    '(GREEN - NIR) / (GREEN + NIR)', {
      GREEN: image.select('B3'),
      NIR: image.select('B5')
    }).rename('NDWI');

  return {
    ndbi: ndbi,
    ndwi: ndwi
  };
}

var indeksKetat = hitungIndeks(medianKetat);
var indeksKustom = hitungIndeks(medianKustom);

// 9. KLASIFIKASI BANGUNAN DENGAN AMBANG YANG SAMA
var bangunanKetat = indeksKetat.ndbi.gt(-0.1)
  .and(indeksKetat.ndwi.lte(0.1));

var bangunanKustom = indeksKustom.ndbi.gt(-0.1)
  .and(indeksKustom.ndwi.lte(0.1));

Map.addLayer(
  bangunanKetat.selfMask(),
  {palette: ['red']},
  'Bangunan -- masking ketat',
  false
);

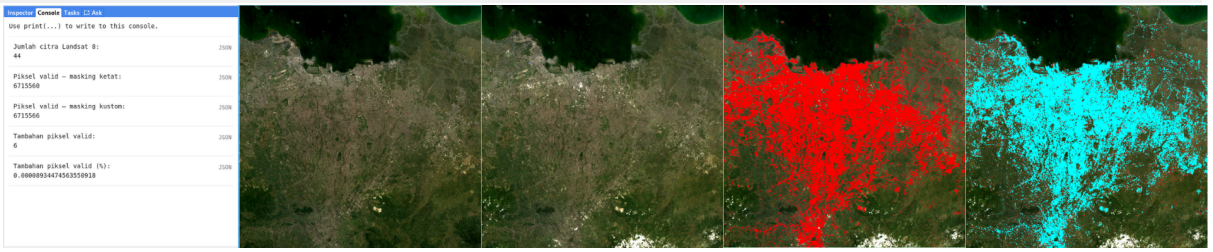
Map.addLayer(
  bangunanKustom.selfMask(),
  {palette: ['cyan']},
  'Bangunan -- masking kustom',
  false
);

// 10. VISUALISASI PERUBAHAN MASKING
// 1 = valid pada kedua metode
```

```
// 2 = valid hanya pada metode kustom
// 3 = valid hanya pada metode ketat
var validKetat = medianKetat.select('B5').mask().gt(0);
var validKustom = medianKustom.select('B5').mask().gt(0);

var statusValid = ee.Image(0)
  .where(validKetat.and(validKustom), 1)
  .where(validKustom.and(validKetat.not()), 2)
  .where(validKetat.and(validKustom.not()), 3)
  .updateMask(validKetat.or(validKustom));

Map.addLayer(
  statusValid,
  {
    min: 1,
    max: 3,
    palette: ['808080', '00FFFF', 'FF00FF']
  },
  'Perbandingan piksel valid',
  false
);
```



membandingkan dua metode masking awan pada citra Landsat 8 tahun 2022 di wilayah Jabodetabekpunjur. Metode pertama menggunakan masking ketat, yaitu membuang piksel apabila terdeteksi awan, bayangan awan, cirrus, atau awan yang melebar. Metode kedua menggunakan masking kustom yang lebih toleran, yaitu membuang piksel hanya jika setidaknya dua dari empat indikator tersebut aktif. Kedua metode kemudian menghasilkan komposit citra yang dapat dibandingkan untuk melihat perbedaan hasil penyaringan awan.

Selanjutnya, kode menghitung jumlah piksel valid pada masing-masing komposit untuk mengetahui apakah metode kustom mempertahankan lebih banyak piksel dibandingkan metode ketat. Hasilnya ditampilkan dalam bentuk jumlah piksel, tambahan piksel, dan persentase penambahannya. Kode juga menampilkan citra berwarna alami dari kedua komposit agar perbedaan visualnya dapat diamati. Dengan demikian, pengguna dapat menilai seberapa besar pengaruh pilihan metode masking terhadap jumlah data citra yang masih dapat digunakan.

Terakhir, kode menghitung indeks NDBI dan NDWI dari kedua komposit, lalu menggunakannya untuk mengidentifikasi kandidat wilayah terbangun dengan ambang yang sama. Hasil klasifikasi bangunan dari kedua metode ditampilkan pada peta, disertai peta perbandingan status piksel valid yang menunjukkan lokasi piksel yang valid pada kedua metode, hanya pada metode kustom, atau hanya pada metode ketat. Secara umum, kode ini digunakan untuk mengevaluasi kompromi antara penyaringan awan yang ketat dan

mempertahankan lebih banyak data citra, sekaligus mengamati dampaknya terhadap identifikasi wilayah terbangun.

2. Kalibrasi ambangan

```
// ====================================================
// PERBANDINGAN NDWI DAN MNDWI - LANDSAT 8 TAHUN 2022
// ROI: wilayah Jakarta dan sekitarnya
// Ambang air: indeks > 0.1
// Luas dihitung menggunakan ee.Reducer.sum()
// ====================================================

// 1. AREA PENELITIAN
var roi = ee.Geometry({
  geodesic: false,
  type: 'Polygon',
  coordinates: [[
    [106.53598136542733, -6.6221756519036195],
    [107.23910636542733, -6.6221756519036195],
    [107.23910636542733, -5.928728445061094],
    [106.53598136542733, -5.928728445061094],
    [106.53598136542733, -6.6221756519036195]
  ]]
});

Map.centerObject(roi, 10);
Map.addLayer(roi, {color: 'white'}, 'Batas ROI');

// 2. KOLEKSI LANDSAT 8
var l8 = ee.ImageCollection('LANDSAT/LC08/C02/T1_L2');

var filtered = l8
  .filterBounds(roi)
  .filterDate('2022-01-01', '2023-01-01');

// 3. MASKING AWAN DAN SKALA REFLEKTANSI
function cloudMask(image) {
  var qa = image.select('QA_PIXEL');

  var dilated = 1 << 1;
  var cirrus  = 1 << 2;
  var cloud   = 1 << 3;
  var shadow  = 1 << 4;

  var mask = qa.bitwiseAnd(dilated).eq(0)
    .and(qa.bitwiseAnd(cirrus).eq(0))
```

```
    .and(qa.bitwiseAnd(cloud).eq(0))
    .and(qa.bitwiseAnd(shadow).eq(0));

  var bands = image.select(
    ['SR_B3', 'SR_B5', 'SR_B6'],
    ['GREEN', 'NIR', 'SWIR']
  );

  var scaled = bands.multiply(0.0000275).add(-0.2);

  return scaled.updateMask(mask)
    .copyProperties(image, ['system:time_start']);
}

// Komposit median yang sama digunakan oleh kedua indeks
var median = filtered
  .map(cloudMask)
  .median()
  .clip(roi);

print('Jumlah citra Landsat 8:', filtered.size());

// 4. MENGHITUNG NDWI DAN MNDWI
// NDWI  = (Green - NIR) / (Green + NIR)
// MNDWI = (Green - SWIR) / (Green + SWIR)

var ndwi = median.expression(
  '(GREEN - NIR) / (GREEN + NIR)', {
    GREEN: median.select('GREEN'),
    NIR: median.select('NIR')
  }
).rename('NDWI');

var mndwi = median.expression(
  '(GREEN - SWIR) / (GREEN + SWIR)', {
    GREEN: median.select('GREEN'),
    SWIR: median.select('SWIR')
  }
).rename('MNDWI');

// 5. KLASIFIKASI AIR DENGAN AMBANG YANG SAMA
var threshold = 0.1;

var waterNDWI = ndwi.gt(threshold).rename('Air_NDWI');
var waterMNDWI = mndwi.gt(threshold).rename('Air_MNDWI');
```

```
// 6. MENGHITUNG LUAS AIR (HEKTARE)
// Pixel area dikonversi dari m² ke hektare.
// Reducer.sum menjumlahkan luas semua piksel yang
// memenuhi kriteria air.

var pixelAreaHa = ee.Image.pixelArea()
  .divide(10000)
  .rename('area_ha');

var areaNDWI = pixelAreaHa
  .updateMask(waterNDWI)
  .reduceRegion({
    reducer: ee.Reducer.sum(),
    geometry: roi,
    scale: 30,
    maxPixels: 1e13,
    tileScale: 4
  });

var areaMNDWI = pixelAreaHa
  .updateMask(waterMNDWI)
  .reduceRegion({
    reducer: ee.Reducer.sum(),
    geometry: roi,
    scale: 30,
    maxPixels: 1e13,
    tileScale: 4
  });

var luasNDWI = ee.Number(areaNDWI.get('area_ha'));
var luasMNDWI = ee.Number(areaMNDWI.get('area_ha'));

print('Luas air NDWI (Ha):', luasNDWI);
print('Luas air MNDWI (Ha):', luasMNDWI);
print('Selisih luas bertanda (MNDWI - NDWI), Ha:',
      luasMNDWI.subtract(luasNDWI));
print('Selisih luas absolut (Ha):',
      luasMNDWI.subtract(luasNDWI).abs());

// 7. MEMBANDINGKAN KLASIFIKASI PER PIKSEL
// Nilai:
//  1 = hanya NDWI mengidentifikasi air
// -1 = hanya MNDWI mengidentifikasi air
//  0 = kedua indeks sepakat
//
// Perbandingan hanya dilakukan pada piksel yang valid
```

```
// pada kedua indeks.

var difference = waterNDWI
  .subtract(waterMNDWI)
  .rename('Perbedaan');

var disagreement = difference.neq(0)
  .selfMask()
  .rename('Ketidaksepakatan');

Map.addLayer(ndwi, {
  min: -1, max: 1,
  palette: ['brown', 'white', 'blue']
}, 'Nilai NDWI', false);

Map.addLayer(mndwi, {
  min: -1, max: 1,
  palette: ['brown', 'white', 'blue']
}, 'Nilai MNDWI', false);

Map.addLayer(waterNDWI.selfMask(), {
  palette: ['0000FF']
}, 'Air menurut NDWI');

Map.addLayer(waterMNDWI.selfMask(), {
  palette: ['00FFFF']
}, 'Air menurut MNDWI', false);

// Merah = hanya NDWI mendeteksi air
// Cyan = hanya MNDWI mendeteksi air
Map.addLayer(
  difference.updateMask(difference.neq(0)),
  {
    min: -1,
    max: 1,
    palette: ['00FFFF', 'FF0000']
  },
  'Peta selisih NDWI-MNDWI'
);

// 8. IDENTIFIKASI TUTUPAN LAHAN DI WILAYAH BERBEDA
// ESA WorldCover 2021
var worldcover = ee.ImageCollection(
  'ESA/WorldCover/v200'
).first().select('Map').clip(roi);
```

```
// Luas wilayah ketidaksepakatan per kelas tutupan lahan.
// Band pertama = luas (Ha), band kedua = kode kelas.
// Grouped reducer menjumlahkan luas untuk setiap kelas.

var areaByLandcover = pixelAreaHa
  .updateMask(difference.neq(0))
  .addBands(worldcover.rename('kelas'));

var landcoverStats = areaByLandcover.reduceRegion({
  reducer: ee.Reducer.sum().group({
    groupField: 1,
    groupName: 'kode_tutupan_lahan'
  }),
  geometry: roi,
  scale: 30,
  maxPixels: 1e13,
  tileScale: 4
});

print(
  'Luas ketidaksepakatan per kelas WorldCover (Ha):',
  landcoverStats
);

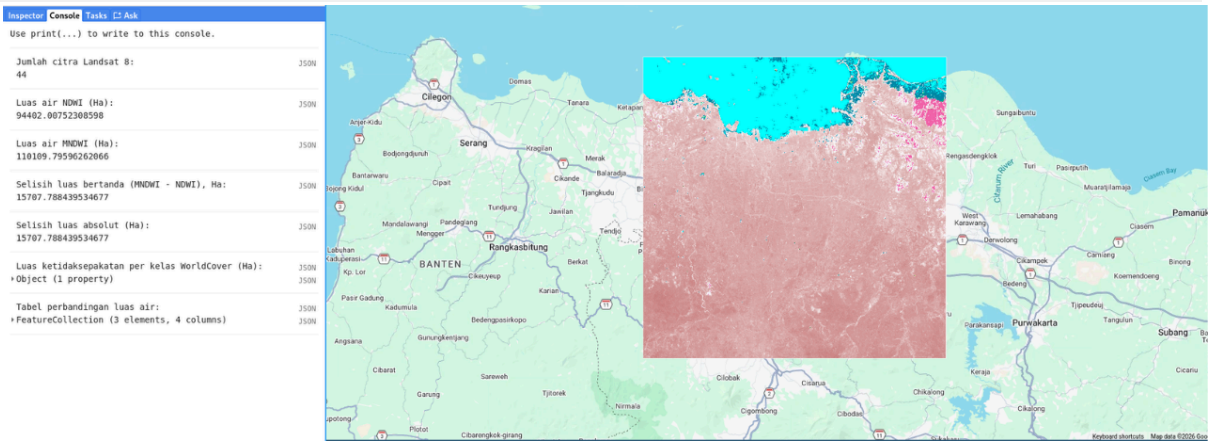
// Kode kelas ESA WorldCover:
// 10 = pohon
// 20 = semak
// 30 = vegetasi herba
// 40 = lahan pertanian
// 50 = kawasan terbangun
// 60 = lahan terbuka / jarang bervegetasi
// 70 = salju/es
// 80 = badan air permanen
// 90 = vegetasi akuatik
// 95 = mangrove
// 100 = lumut dan liken

// Tampilkan tutupan lahan hanya di lokasi berbeda
Map.addLayer(
  worldcover.updateMask(difference.neq(0)),
  {
    min: 10,
    max: 100,
    palette: [
      '006400', // 10 pohon
      'ffbb22', // 20 semak
```

```
      'ffff4c', // 30 vegetasi herba
      'f096ff', // 40 pertanian
      'fa0000', // 50 terbangun
      'b4b4b4', // 60 lahan terbuka
      'f0f0f0', // 70 salju/es
      '0064c8', // 80 badan air
      '0096a0', // 90 vegetasi akuatik
      '00cf75', // 95 mangrove
      'fae6a0'  // 100 lumut/liken
    ]
  },
  'Tutupan lahan pada wilayah berbeda',
  false
);

// 9. RINGKASAN HASIL DALAM TABEL
var comparison = ee.FeatureCollection([
  ee.Feature(null, {
    'Indeks': 'NDWI',
    'Ambang': threshold,
    'Luas_air_Ha': luasNDWI
  }),
  ee.Feature(null, {
    'Indeks': 'MNDWI',
    'Ambang': threshold,
    'Luas_air_Ha': luasMNDWI
  }),
  ee.Feature(null, {
    'Indeks': 'Selisih absolut',
    'Ambang': threshold,
    'Luas_air_Ha': luasNDWI.subtract(luasMNDWI).abs()
  })
]);

print('Tabel perbandingan luas air:', comparison);
```



Menurut Xu (2006), NDWI dapat menghasilkan built-up land noise: sebagian permukaan terbangun memiliki respons spektral yang membuatnya ikut teridentifikasi sebagai air. Akibatnya, luas air bisa terestimasi terlalu besar. MNDWI mengganti band NIR pada NDWI dengan band SWIR, yang membantu menekan respons permukaan terbangun serta gangguan dari tanah dan vegetasi, sehingga air terbuka lebih mudah dibedakan dari latar perkotaan.

Namun, MNDWI tidak otomatis selalu lebih akurat untuk semua lokasi. Hasil akhirnya tetap dipengaruhi ambang 0,1, kondisi air, bayangan bangunan, awan yang tersisa, dan resolusi citra. Kelas WorldCover yang tumpang tindih dengan wilayah berbeda menunjukkan konteks tutupan lahan, bukan bukti pasti bahwa suatu piksel salah diklasifikasikan. Untuk membuktikan keunggulan akurasi, hasilnya perlu dibandingkan dengan data referensi air yang tervalidasi.